Exercise Sheet 4 – Vector Clocks

Lab on 3.12.25 and 5.12.25

In this lab you will implement a vector clock and add it to our message board system. You first implement the core logic of the vector clock as a class. Next, you extend `AsyncBoardProxy.py` and the stub in `BoardServer.py` to add the time to every message exchanged between servers as needed for vector clocks. Finally, you integrate the vector clock into our message board application. When a server receives a new message from a client (with `put()`), it adds a time stamp to the message and forwards the extended message to all other servers. Each server sorts the messages by the time stamps to enable that clients see the same order of messages on each server.

You have to submit your solutions by showing them to your tutor. The latest submission is possible in the lab, in which the next exercise sheet is presented (see table below). You can also submit in the lab with the question time on the exercise sheet.

	Presentation of the tasks	Question time	Last possible submission
Group 1:	Wednesday, 03.12.25	Wednesday, 10.12.25	Wednesday, 17.12.25
Group 2:	Friday, 05.12.25	Friday, 12.12.25	Friday, 19.12.25

For the tasks of this exercise sheet you get 8 points in total. You get it for:

	Task 1	Task 2	Task 3	Task 4		
Points:	2	2	3	1		

Task 1 Implement Vector Clock Class

Implement the core of a vector clock as a class. Make sure that you have understood vector clocks. Repeat the slides from the lecture or read in the book (van Steen / Tanenbaum: Distributed Systems) if necessary.

A class with empty methods is given in file `VectorClock.py` that you find in StudIP in `DSLabs4_GivenFiles.zip`. Download the Zip file and add its files to the server files of your previous lab. `VectorClock.py` contains the class clock modelling a vector clock. It needs an attribute to store the vector of a server's clock as list initialized with zeros. Method `eventHappens()` triggers an event on the server increasing its value in the vector. Method `getTime()` returns the vector and also triggers an event as common for vector clocks. The method `getTimeNoEvent()` does the same, but without triggering the event. Method `updateTime(timeFromOtherServer)` updates the local time after receiving a time timestamp from another server. The time stamp vector is given to the method as list.

The file `VectorClock.py` also contains functions for comparing two times for equality, causality, and order. These are just functions, not methods of the class. Times are given to it as parameters that are lists as returned by `getTime()` and `getTimeNoEvent()`. Function `equal(t1, t2)` tests if two times are equal. Function `smallerEqual(t1, t2)` tests if time t_1 is causally before t_2. Function `concurrent(t1, t2)` returns True only if neither t_1 is causally before t_2 nor t_2 before t_1. Function `totalOrder(t1, t2)` compares two times to define a unique total order. The functions return a boolean.

The goal of function `totalOrder()` is to get times of vector clocks into a total order as common for times. It brings all times into a one-dimensional unique order that we will use to sort messages according to its time stamps. The function gets two times t_1 and t_2 as parameters and returns -1 if $t_1 < t_2$, 0 if $t_1 == t_2$, and 1 if $t_1 > t2$. The causal order of timestamps should be tested with the functions `equal()`, `smallerEqual()`, and `concurrent()`. If two timestamps are concurrent (have not causal order) but not equal (e.g. `[2,3,0,1]` and `[4,3,2,1]`), define any order that is the same on all servers.

Test your class by starting `VectorClockTest.py`. It imports your file `VectorClock.py`, creates 3 objects, and creates events by simulating a message exchange between three servers. The scenario is shown in Figure 1. To test your code, check that the program outputs are the same times as shown in the figure. Reason if the causalities (concurrent, smaller or equal) that are shown are correct. Figure and scenario were taken from Figure 5.12 of the book Maarten van Steen, Andrew S. Tanenbaum: Distributed Systems, Fourth edition (4.03), self-published, January 2025.

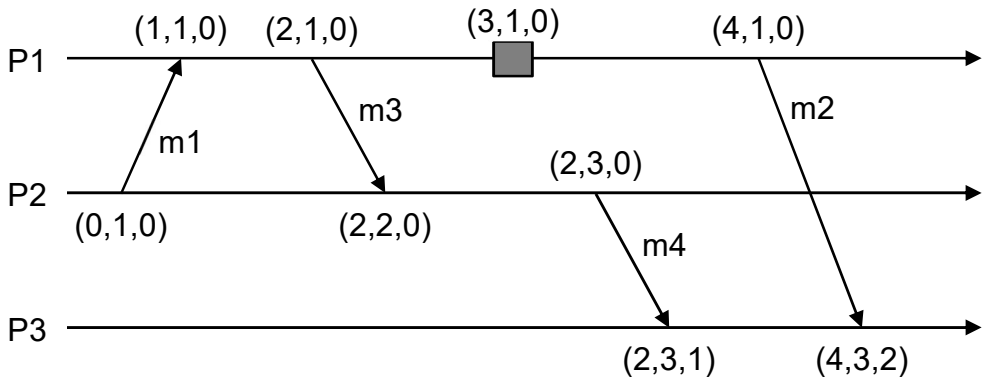

Figure 1: Scenario implemented in VectorClockTest.py (from Steen / Tanenbaum: Distributed Systems)

Task 2 Add time stamps to proxy and stub

Change your proxy and stub to include time stamps in each message exchanged between the servers. When a message is sent (in proxy or stub) the current time is retrieved from the vector clock with `getTime()` also increasing the server's time value in the vector clock. When a message is received, the contained time stamp is extracted and given to the method `updateTime()` to update the local vector clock.

Add the time to each message. Each JSON message is a dictionary (in JSON called object). You add the time as additional element (name-value pair), e.g. `"TIME":[10, 14, 4, 6]`.

Change the protocols so that all messages between the servers are JSON messages. Unfortunately, the protocol used for the previous labs used JSON messages only for requests. Values such as strings, integers, or lists were used for responses. You need to change their format to JSON messages to be able to add the time vectors. Use a format in which an element "RESULT" indicates success or error (e.g. `"RESULT":"OK"` or `"RESULT":"ERROR"` or `"RESULT":"UNKNOWN_INDEX"`). Other elements are used to return a message (`get()`), the number of messages (`getNum()`) or the entire message board (`getBoard()`). Also update the methods used for mutexes and leader election.

As your servers now always send responses as JSON messages, the proxy of your client (`BoardProxy.py`) will have problems receive it, as it is not implemented for that. You can solve this issue in two ways. You can implement the stub in your server to return responses to the client still in the old format (not JSON). Or you can also update the client proxy (`BoardProxy.py`) for the new format (responses as JSON).

The vector clock object is created in the `…_Main.py` file and given to other modules of your server. The constructor in `AsyncBoardProxy.py` gets the object as its fourth parameter `logicalClock`. Your function `startServer()` in `BoardServer.py` gets it as its sixth parameter `logicalClockParam`. Both parameters should have a default value of `None`. Your code should also work correctly if it is `None` to support server versions from previous labs. Refer to the files `UseMutexForUpdatesWithClock_Main.py` and `InformAllOtherServersWithClock_Main.py` both contained in `DSLabs4_GivenFiles.zip` from StudIP for object creation. The first will start a server from exercise 3 for testing. The second is used in Task 3 of this exercise.

Test your code with the server cluster from exercise 3. Start each server with `UseMutexForUpdatesWithClock_Main.py` that creates a vector clock object and passes it to `AsyncBoardProxy.py` and `BoardServer.py`. Use the logging to observe the messages exchanged between the servers. Do all messages contain time stamps? Are the time stamps reasonable in the sense of a vector clock?

Task 3 Integrate Vector Clock to Message Board System

Integrate the vector clock into the message board system. Start with the algorithm in which each server informs each other server about changes (`InformAllOtherServers.py`) from exercise 2. With that algorithm the servers return the messages in different orders. To get the messages into the same order, each message is given a timestamp from the vector clock. Each server sorts the messages by their time stamps. The server that receives a new message from a client adds a timestamp to the message before it saves it locally and forwards it to the other servers.

a) Add a timestamp to each message

Copy your file `InformAllOtherServers.py` into the new file `InformAllOtherServersWithClock.py`. Implement the algorithm in the new file. The constructor of its class `storage` gets an object of your vector clock as fifth parameter. You need to declare this parameter. The vector clock object is created and passed to your object by `InformAllOtherServersWithClock_Main.py` that starts a server.

Extend method `put()` to add the timestamp to a message. To do so, when a message is received from a client, replace the message (a string) by a list consisting of the time stamp and the message (e.g. `[[9, 2, 2, 2], "Hallo"]`). The list is further used as message, that is, it is stored in your `AsyncBoardStorage` and forwarded to the other servers. Make sure, that no timestamp is added, when `put()` is called by another server, as the timestamp is already contained in the message.

In method `modify()`, only change the message but preserve the value of the old timestamp. E.g. if `modify()` is called to change message `[[9, 2, 2, 2], "Hallo"]` into `"Hi"`, all server have the message `[[9, 2, 2, 2], "Hi"]` afterwards. This leads to the conceptional issue, that users can update a message to an answer another message

with a bigger timestamp. Hence, the answer has a lower timestamp than the question, which should not be the case in our message board. You may ignore this issue or develop an algorithm solving it.

Correct all server and client code, if it has problems with the new messages. Errors may occur, because a message is now a list from a time stamp and a string, and no more just a string. In `UserInterface.py`, messages should be displayed as the list with time stamp and string to allow you as user to observe the timestamps.

Test your software. Start the four servers using `InformAllOtherServersWith-Clock_Main.py`. Do you see the timestamps of messages in `UserInterface.py` when using the commands `get` and `getboard`? Do the timestamps remain the same, when calling `modify`? Do you see the same timestamps when connecting to different servers?

b) Sort messages by timestamp

Two steps are necessary to sort messages by their timestamps. First, you need to implement a mechanism for sorting messages in `AsyncBoardStorage.py`. Second, you need to provide a function that defines the desired order of the messages. This is necessary, as `AsyncBoardStorage.py` has no understanding of messages and its time stamps and should remain independent of it to be applicable to all types of messages.

The definition of the correct order of messages should be defined in the module of the message distribution algorithm (in `InformAllOtherServersWithClock.py`). The order must be identical on all servers, so that all servers return the messages in the same order. It should also reflect the causal ordering of the vector clock's time stamps to ensure, that a message with a question is shown before the message with its answer.

Define the ordering as function `compareMessages()` that gets two messages m_1 and m_2 as parameters and returns -1 if $m_1 < m_2$, 0 if $m_1 == m_2$, and 1 if $m_1 > m_2$. Bear in mind, that a message is a list, in which the timestamp of the vector clock is the first element. Call function `totalOrder()` from `VectorClock.py` to do the actual comparison. Define function `compareMessages()` (without `async`) as class method of class `storage` in `InformAllOtherServersWithClock.py` with the following signature:

```
def compareMessages(m1, m2):
    …
```

The module `AsyncBoardStorage.py` needs to call the function `compareMessages()` to be able to sort messages according to it. For that purpose, the constructor of the class `storage` in `AsyncBoardStorage.py` gets the function as parameter. This is implemented in the given file `InformAllOtherServersWithClock_Main.py`. But you have to declare the parameters of the constructor. Give it a default value of `None` so that the old server versions still work. If it is `None`, do not sort.

It is assumed, that the messages in `AsyncBoardStorage` are stored in as list. A list can be sorted with its method `sort()`. Its named parameter `key` allows defining the ordering used for sorting. The module `functools` provide a function `cmp_to_key()` that allows using your function `compareMessages()` as such key. If we (incorrectly) assume that the list of messages is stored in variable X and we could directly access the function `compareMessages()`, we could sort the list with the following code:

```
import functools
X.sort(key=functools.cmp_to_key(comperatorForElements))
```

Task 4 Measurements and Experiments

Repeat the experiments from the previous exercise sheets.

a) Use the script `SendDataToSeveralServers.py` to upload 4 messages concurrently to each server and observe their message boards. Are there the same 16 messages on each server? Are these in the same order? If not, your program from Task 3 has an error. (Correct it!) Take notes about the results you have observed.

   The program `SendDataToSeveralServers.py` might show an error when displaying messages, as your proxy returns a message no more as a string, but as a list consisting of the timestamp and the message string. Change the program so that it can display the messages with times.

b) Use the script `SendDataToSeveralServers.py` to measure the uploading time when concurrently uploading 1000 messages to each of the four servers. Compare your results to those of the previous exercises. Write down the time for uploading the messages and your observations.